REALTIME: WEBSOCKETS+SSE+RXJS

Замовлення №42: polling → SSE → WS → Socket.IO → Redis → RxJS

Заняття №18 | Node.js Pro

Лектор
Кирило Орлов

ЗАМОВЛЕННЯ №42: КЛІЄНТ ХОЧЕ ЗНАТИ ПЕРШИМ

Частина 1 — драбинка транспортів	Частина 2 — канал стає продуктом
кроки 1-2 · polling: 6 із 7 — порожні	крок 6 · socket.io: кімнати + ack + reconnect
крок 3 · SSE: reconnect + Last-Event-ID	крок 7 · 2 інстанси + Redis adapter
крок 4 · WS handshake руками (той самий upgrade!)	крок 8 · Nest gateway + RxJS backpressure
крок 5 · бібліотека ws: broadcast	односпрямований потік — SSE; двосторонній — WS

НАСКРІЗНА НИТКА

Одна подія на весь вечір: «замовлення 42 → SHIPPED». Питання одне: як вона доїде до клієнта раніше, ніж він натисне F5? Кожен крок — окремий процес на портах 21801–21812; усі виводи в деці — з реальних запусків code/18_Realtime.

ПЛАН ЗАНЯТТЯ



- Чому HTTP мовчить: ціна polling
- Long-polling: latency ≈ 0
- SSE: push сервер→клієнт, reconnect
- WebSocket: handshake руками, фрейми
- ws і Socket.IO: кімнати, ack
- Два інстанси: кімнати і Redis adapter
- NestJS gateway + RxJS · ДЗ

СЬОГОДНІ ПОБАЧИТЕ НА ВЛАСНІ ОЧІ

01. 6 із 7

сім запитів за сім секунд — шість порожніх: ціна short polling

02. Last-Event-ID

сервер рве з’єднання кожні 3 події — клієнт отримує 8 без дублів і дірок

03. 101 Switching

наш ручний сервер на 60 рядків домовляється зі справжнім WS-клієнтом

04. кімнати

Ann у orders:42 чує notification, Carl у сусідній кімнаті — тишу

05. розрив кімнати

два інстанси: без адаптера Boris мовчить, з Redis Pub/Sub — чує

06. 40/40 · 3/40 · 1/40

три стратегії повільного споживача: черга, дросель, скасування

ПРОБЛЕМА: У HTTP ПЕРШИМ ГОВОРИТЬ КЛІЄНТ

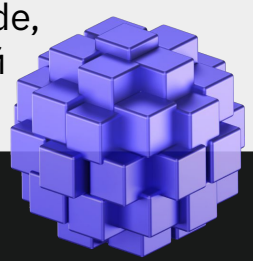
запит → відповідь → кінець: ініціатива завжди у клієнта (L#3)

«замовлення 42 спаковане» сервер сам сказати НЕ МОЖЕ

перша ідея: питати частіше — setInterval + GET /poll

у проді це нотифікації, дашборди, прогрес, чат, LLM streaming

Сьогодні без браузера і моків: сервери — чистий Node, клієнти — curl і короткі Node-скрипти. Спершу сирий транспорт, потім бібліотека.



```
ХТО ГОВОРИТЬ ПЕРШИМ?

клієнт      сервер
──────      ──────
GET /poll →
       ← []
GET /poll →
       ← []
  (подія №1!)
GET /poll →
       ← [подія]

──────────────────
подія лежала в
черзі, поки клієнт
не спитав. Latency
= інтервал polling
```

КРОКИ 1-2 · POLLING: КОРОТКИЙ І ДОВГИЙ

```
// stages/02-long-polling.mjs — сервер ТРИМАЄ запит
const waiters = new Set();     // відкриті відповіді
setInterval(() => {
  events.push({ id: nextId++, type: 'order.status' });
  for (const w of waiters) w.wake();   // будимо всіх
}, 5000);

const server = createServer((req, res) => {
  const after = Number(url.searchParams.get('after') ?? 0);
  const answer = () => res.end(JSON.stringify(
    events.filter((e) => e.id > after)));
  if (events.some((e) => e.id > after)) return answer();
  // немає — «підвішуємо» відповідь до події/таймауту:
  const waiter = { wake: () => { cleanup(); answer(); } };
  const timer = setTimeout(() => { cleanup(); answer(); },
    HOLD_MS);              // 25 с — менше таймаутів проксі
  const cleanup = () => { clearTimeout(timer);
    waiters.delete(waiter); };
  waiters.add(waiter);
  req.on('close', cleanup);  // клієнт пішов — прибери
});
```

Сценарій: подія раз на 5 с, клієнт хоче знати одразу

1 · GET /poll?after=N — курсор останньої баченої події

2 · крок 1: питаємо щосекунди — 6 із 7 відповідей порожні

3 · long-poll: подій нема — відповідь «підвисає» у waiters

вивід: запит #1 висів 4851 мс → 1 подія, порожніх нуль

урок: latency ≈ 0, але кожна подія = повний HTTP-цикл

ГАЗЕТА, ПОШТА І ТЕЛЕФОН: ТРАНСПОРТИ ЯК АНАЛОГІЯ

ТРАНСПОРТ	АНАЛОГІЯ (ПОШТА)
• short polling — питаємо щосекунди • long-polling — запит чекає на подію • SSE — потік подій в один бік • WebSocket — двосторонній канал • socket.io — кімнати + reconnect	• сам бігаєш на пошту щодня • стоїш біля віконця, поки не прийде • підписка: газету кидають під двері • телефонна лінія: чути обох • секретар: передзвонить і з’єднає

Де аналогія ламається: газета (SSE) не понесе відповідь назад — писати в редакцію доведеться звичайною поштою (POST-запитом поруч). А телефонна лінія (WS) після обриву сама не передзвонить: reconnect — чиясь робота. У SSE це робить EventSource, у WS — ти сам або socket.io.



КРОК 3 · SSE: PUSH ЗА ЦІНОЮ ЗВИЧАЙНОГО HTTP

- нескінченна HTTP-відповідь: content-type: text/event-stream
- формат: id: / event: / data: + порожній рядок-роздільник
- reconnect робить EventSource — нашого коду в ньому нуль
- Last-Event-ID: сервер догрався з буфера — 8 подій без дублів
- retry: 1500 — сервер сам підказує клієнту темп reconnect
- межа: тільки сервер→клієнт. Двосторонній канал — крок 4

```
SSE ОЧИМА CURL · РЕАЛЬНИЙ ПРОГІН КРОКУ 3
$ curl -sN :21803/events
retry: 1500

id: 1
event: order.status
data: {"id":1,"status":"SHIPPED"}

id: 2
event: order.status
data: {"id":2,"status":"PACKED"}
```

сервер рве з'єднання кожні 3 події; EventSource повертається САМ із заголовком Last-Event-ID: 3 → 6. Клієнт отримав 8/8 — без дублів



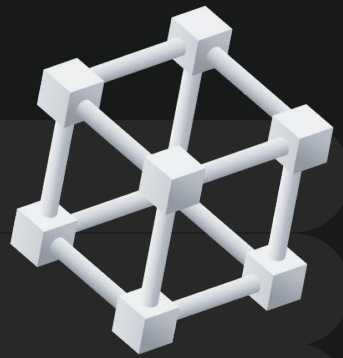
```
// stages/03-sse.mjs — потік + резюме з Last-Event-ID
const history = [];            // буфер останніх подій
setInterval(() => {
  const e = { id: nextId++,
    status: nextId % 2 ? 'PACKED' : 'SHIPPED' };
  history.push(e);
  for (const client of clients) client.push(e);
}, 1000);
const server = createServer((req, res) => {
  // реконект? EventSource САМ ставить цей заголовок:
  const lastSeen = Number(req.headers['last-event-id'] ?? 0);
  res.writeHead(200, { 'content-type': 'text/event-stream',
    'cache-control': 'no-cache' });
  res.write('retry: 1500\n\n'); // темп реконекту клієнта
  const push = (e) => res.write(
    `id: ${e.id}\nevent: order.status\n` +
    `data: ${JSON.stringify(e)}\n\n`);
  // спершу — все, що клієнт пропустив офлайн:
  for (const e of history.filter((e) => e.id > lastSeen))
    push(e);
  clients.add({ push });
});
// лог: «клієнт підключився, Last-Event-ID: 3» → віддав 4,5,6
```



```
// stages/04-ws-handshake.mjs — рукостискання без бібліотек
const MAGIC = '258EAFA5-E914-47DA-95CA-C5AB0DC85B11';
// ↑ константа RFC 6455 — однакова для всіх серверів світу
server.on('upgrade', (req, socket) => {  // той самий L#3!
  const key = req.headers['sec-websocket-key'];
  const accept = createHash('sha1').update(key + MAGIC).digest('base64');
  socket.write(
    'HTTP/1.1 101 Switching Protocols\r\n' +
    'Upgrade: websocket\r\nConnection: Upgrade\r\n' +
    `Sec-WebSocket-Accept: ${accept}\r\n\r\n`);
  // після цього рядка HTTP закінчився — далі фрейми:
  socket.on('data', (buf) => {
    const opcode = buf[0] & 0x0f;     // 0x1 text · 0x8 close
    const masked = (buf[1] & 0x80) !== 0; // клієнт МУСИТЬ
    const len = buf[1] & 0x7f;        // до 125 — навчально
    const maskKey = buf.subarray(2, 6);
    const text = Buffer.from(buf.subarray(6, 6 + len)
      .map((b, i) => b ^ maskKey[i % 4])).toString('utf8');
    // ← фрейм: FIN=true opcode=0x1 MASK=true len=33
  });
}); // curl → 426 Upgrade Required · WS-клієнт Node 22 → 101
```



ЧОТИРИ ТРАНСПОРТИ: ЩО КУПУЄШ І ЩО ПЛАТИШ

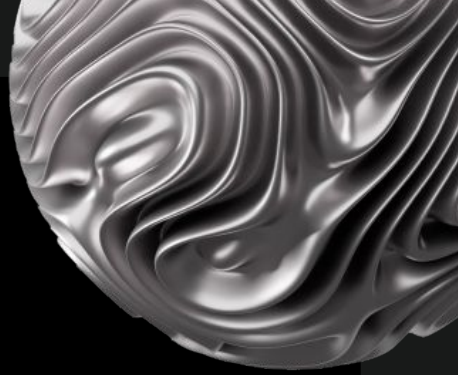


short polling — просто, cache-friendly; ціна: марні запити, latency = інтервал

long-polling — latency ≈ 0 на тому ж HTTP; ціна: цикл на кожну подію

SSE — push + reconnect + resume задарма; ціна: лише сервер→клієнт, текст

WebSocket — full-duplex, фрейм від 2 байтів заголовка; ціна: Upgrade не люблять деякі проксі, reconnect і кімнати — руками. Правило вечора: односпрямований потік — почни з SSE; двосторонній — WebSocket. Polling сьогодні — fallback або cache-friendly опитування раз на хвилини



```
// stages/05-ws-broadcast.mjs — бібліотека ws: broadcast
wss.on('connection', (ws, req) => {
  ws.on('message', (data) => {
    for (const client of wss.clients)     // «кімнат» нема:
      if (client.readyState === WebSocket.OPEN)
        client.send(`${name}: ${data}`);
  });
});
setInterval(() => {               // heartbeat — теж руками
  for (const client of wss.clients) client.ping(); }, 10_000);
// stages/06-socketio.mjs — те саме + кімнати + ack:
io.on('connection', (socket) => {
  socket.on('join', (room, ack) => {
    socket.join(room);            // кімната = мітка на сокеті
    ack(`ви в кімнаті ${room}`); // вбудований req/response
  });
  socket.on('notify', ({ room, text }) =>
    io.to(room).emit('notification', { room, text }));
});
// лог: transport=polling → upgrade · рестарт: reconnect_attempt #1..#3
```

КРОК 6 · SOCKET.IO: ЩО ВІН ДОДАЄ ПОВЕРХ WS

rooms

кімната — server-side мітка на сокеті: join / io.to(room).emit — адресна розсилка без перебору клієнтів

ack

третій аргумент події — callback: вбудований request/response поверх подій; у демо ack підтверджує вхід у кімнату

reconnect

reconnect_attempt #1..#3 → успіх — клієнт сам; heartbeat ping 25 с / timeout 20 с — мертві з’єднання сервер закриє за ~45 с

Socket.IO — не «WebSocket з конфігом»
Це власний протокол (Engine.IO) поверх WS або long-polling: у лозі демо перше підключення — transport=polling, і лише потім upgrade. Драбинка кроків 2→4 зашита в саму бібліотеку — тому вона й переживає корпоративні проксі.

ТАБЛИЦЯ ВЕЧОРА: WS vs SSE vs POLLING

Транспорт	Що дає	Коли брати
short polling	простота; звичайний HTTP і кеш	рідкі події, legacy, fallback
long-polling	latency ≈ 0 без нових протоколів	fallback: socket.io ним стартує
SSE / EventSource	push + reconnect + Last-Event-ID	стрічки, прогрес, LLM streaming
WebSocket (ws)	full-duplex, фрейми від 2 байтів	чат, гра, курсори, спільна робота
Socket.IO	rooms + ack + reconnect + fallback	продові кімнати; 2+ інстанси → адаптер

ДЕ REALTIME ЖИВЕ У ВАШОМУ КУРСОВОМУ

01. нотифікації про статус замовлення — кімната orders:<id> — це ДЗ

02. прогрес довгої операції: імпорт каталогу, генерація звіту — SSE

03. LLM streaming відповіді токен за токеном — SSE (це просто HTTP)

04. чат підтримки, спільні курсори, гра — WebSocket / socket.io

05. живий дашборд адмінки: замовлення в реальному часі — SSE або io

КАРТА РІШЕНЬ: ВІД ЗАДАЧІ ДО ТРАНСПОРТУ

подія раз на хвилини	→	short polling + кеш
потік лише сервер→клієнт	→	SSE (EventSource) + Last-Event-ID для resume
двосторонній канал	→	WebSocket (ws)
кімнати, ack, reconnect	→	socket.io
2+ інстанси за load balancer	→	+ Redis adapter
gateway у Nest-проєкті	→	@nestjs/websockets
продюсер швидший за клієнта	→	RxJS: concatMap / throttleTime / switchMap
двосторонність не потрібна	→	НЕ тягни WS «на виріст»

── ТУТ ЗАКІНЧУЄТЬСЯ ТРАНСПОРТ, ПОЧИНАЄТЬСЯ ПОТІК ─────



КРОК 7 · ДВА ІНСТАНСИ: ПРОГІН ДЕМО С

Сценарій	**Alla → інстанс A :21811**	**Boris → інстанс B :21812**
обидва в кімнаті orders:42	ack: «інстанс A: ви в кімнаті»	ack: «інстанс B: ви в кімнаті»
notify БЕЗ адаптера	notification отримала	ТИША — кімната в пам’яті A
notify З Redis-адаптером	notification отримала	notification отримав — Pub/Sub
що пересилає адаптер	emit-и між інстансами	стан сокетів — НЕ пересилає
що ще треба у проді	sticky sessions на load balancer	бо handshake+polling = 2+ запити

```
// stages/07-scale-redis.mjs — 2 інстанси, один Redis
const io = new Server(PORT);     // 21811 (A) · 21812 (B)
if (USE_ADAPTER) {            // ADAPTER=0 — показати біль
  const pub = createClient(
    { url: 'redis://127.0.0.1:21879' });
  const sub = pub.duplicate(); // Pub/Sub = 2 з'єднання
  await Promise.all([pub.connect(), sub.connect()]);
  io.adapter(createAdapter(pub, sub)); // ОДИН рядок різниці
}
socket.on('notify', ({ room, text }) =>
  io.to(room).emit('notification', { room, text }));

// БЕЗ адаптера (ADAPTER=0):
//   [A:21811] notify → кімната «orders:42»
//   Boris на B: …тиша — кімната живе в пам'яті A
// З адаптером:
//   [Boris] ← notification: де моє замовлення?
// адаптер пересилає ПОДІЇ, а не стан: свої кімнати кожен
// інстанс тримає сам — Pub/Sub тут лише пошта.
// Глибокий Redis (і чому саме Pub/Sub) — L#23
```



КРОК 8 · NEST GATEWAY: ТОЙ САМИЙ SOCKET.IO + DI

@WebSocketGateway() — це provider: DI, guards, pipes — увесь L#7

@SubscribeMessage('join') + @MessageBody() — handler як контролер

повернув WsResponse — Nest сам відповість подією «joined»

повернув Observable — Nest шле кожен елемент, відписка на complete

у демо: interval(50) → throttleTime(500) → take(6): 6 із ~51 тіка

збірка tsc → node dist: tsx не емітить decorator metadata (L#6)

RXJS · ПОВІЛЬНИЙ СПОЖИВАЧ: ЦИФРИ ПРОГОНУ

СТРАТЕГІЯ	ЕФЕКТ (40 подій/5 мс · споживач 50 мс)
• concatMap — черга: обробити ВСЕ	• 40/40 подій · макс. лаг 1829 мс
• throttleTime — дросель: перший у вікні	• 3/40 · лаг 52 мс — ідеал для дашборда
• switchMap — скасування: тільки останнє	• 1/40 · лаг 53 мс — типовий autocomplete
• take(6) — потік, що сам закінчується	• complete → Nest відписався, витоків нуль
• Observable ≠ Promise: багато значень	• push-модель: next / complete / error

```
// stages/08-nest-rxjs/src/events.gateway.ts
@SubscribeMessage('ticker')
ticker(@MessageBody() windowMs: number):
  Observable<WsResponse<{ seq: number }>> {
  return interval(50).pipe(       // продюсер: 20 тіків/с
    throttleTime(windowMs || 500),// споживачу: 2 на секунду
    take(6),                      // і потік скінченний
    map((i) => ({ event: 'tick', data: { seq: i } })),
  );
}
// клієнт: tick #1 seq=0 … tick #6 seq=50 — 6 із ~51
// Потік завершився: take(6) → complete → Nest відписався

// dist/rx-backpressure.js — три стратегії, один прогін:
// concatMap    (черга)   : 40/40 | лаг 1829 мс | 2056 мс
// throttleTime (дросель) :  3/40 | лаг   52 мс |  264 мс
// switchMap    (останнє) :  1/40 | лаг   53 мс |  278 мс
//
// «повільний споживач» не має правильної відповіді за
// замовчуванням: черга, дросель чи скасування —
// це РІШЕННЯ про дані, а не технічна дрібниця
```

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

браузерний UI: сьогодні клієнти headless — curl і Node-скрипти; у ДЗ живе демо можна на двох вкладках. WebRTC і WebTransport — лише терміни у vault; GraphQL subscriptions були на L#17

нутрощі Redis Pub/Sub і кластер → L#23 · черги і гарантії доставки (RabbitMQ) → L#19 · JWT у handshake → L#24 · rate limiting з'єднань → L#27. Сьогодні чесно: один Redis-контейнер для адаптера і жодної автентифікації

Версії демо: Node v22.21.1 · ws 8.21.2 · socket.io 4.8.3 · @nestjs/* 11.1.28 · rxjs 7.8.2 · typescript 6.0.3 · eventsource 4.1.0 · redis:7-alpine. Таймінги і лаги на іншій машині інші — порядок ефекту той самий



ПІДСУМКИ + МІСТ ДО L#19

ТРАНСПОРТИ

polling → long-poll → SSE → WS: кожен щабель руками; односпрямований — SSE, двосторонній — WS

NEST + RXJS

gateway = provider з DI; handler повертає Observable; черга / дросель / скасування — рішення про дані

ДАЛІ

L#19 — RabbitMQ: «order placed» їде в чергу з гарантіями; споживач черги емітить у ваші кімнати

ДЗ #18

7 балів (PROJECT MEDIUM) · дедлайн 08.10 · курсовий: WS-кімнати нотифікацій після перевірки власності + SSE-варіант із Last-Event-ID + headless-демо + таблиця trade-offs

SOCKET.IO

rooms + ack + reconnect + fallback з коробки; на 2+ інстансах кімнати рвуться — лікує Redis-адаптер одним рядком

Q&A



r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ